

CS230: Deep Learning

Initialization & Regularization

Section 4

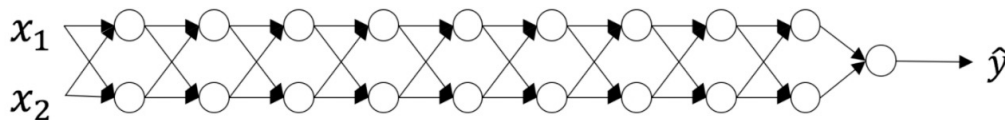
Agenda

1. Initialization Intuition
2. Initialization Goals
3. Xavier + He Initialization
4. Overfitting & Underfitting
5. L1 + L2 Regularization
6. Dropout

Initialization Intuition

II The problem of exploding or vanishing gradients

Consider this 9-layer neural network.



At every iteration of the optimization loop (forward, cost, backward, update), we observe that backpropagated gradients are either amplified or minimized as you move from the output layer towards the input layer. This result makes sense if you consider the following example.

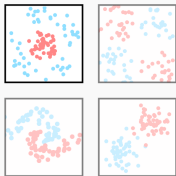
Assume all the activation functions are linear (identity function). Then the output activation is:

$$\hat{y} = a^{[L]} = W^{[L]}W^{[L-1]}W^{[L-2]} \dots W^{[3]}W^{[2]}W^{[1]}x$$

Initialization Intuition

1. Choose input dataset

Select a training dataset.



This legend details the color scheme for labels, and the values of the weights/gradients.

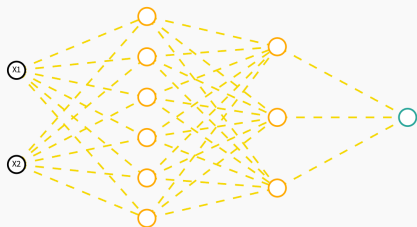


Node Type: ☐ Input ☐ Relu ☐ Sigmoid

2. Choose initialization method

Select an initialization method for the values of your neural network parameters¹.

☐ Zero ☐ Too small ☐ Appropriate ☐ Too large

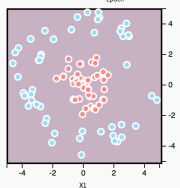
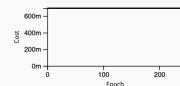


Select whether to visualize the weights or gradients of the network above.

☐ Weight ☒ Gradient

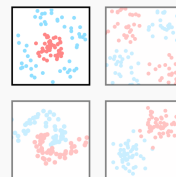
3. Train the network.

Observe the cost function and the decision boundary.



1. Choose input dataset

Select a training dataset.



This legend details the color scheme for labels, and the values of the weights/gradients.



Node Type: ☐ Input ☐ Relu ☐ Sigmoid

2. Choose initialization method

Select an initialization method for the values of your neural network parameters¹.

☐ Zero ☒ Too small ☐ Appropriate ☐ Too large

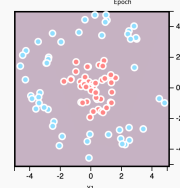
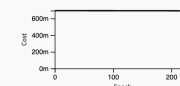


Select whether to visualize the weights or gradients of the network above.

☐ Weight ☒ Gradient

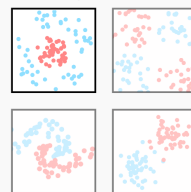
3. Train the network.

Observe the cost function and the decision boundary.



1. Choose input dataset

Select a training dataset.



This legend details the color scheme for labels, and the values of the weights/gradients.



Node Type: ☐ Input ☐ Relu ☐ Sigmoid

2. Choose initialization method

Select an initialization method for the values of your neural network parameters¹.

☐ Zero ☐ Too small ☒ Appropriate ☐ Too large

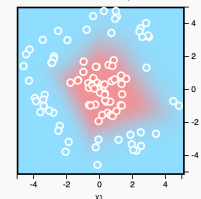
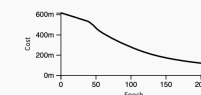


Select whether to visualize the weights or gradients of the network above.

☐ Weight ☒ Gradient

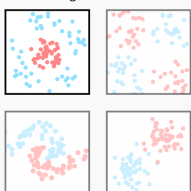
3. Train the network.

Observe the cost function and the decision boundary.



1. Choose input dataset

Select a training dataset.



This legend details the color scheme for labels, and the values of the weights/gradients.

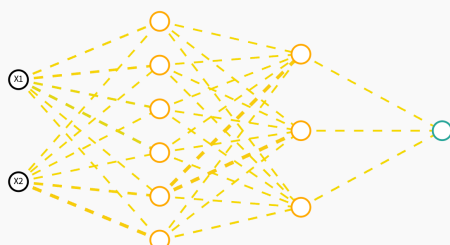


Node Type: ☐ Input ☐ Relu ☐ Sigmoid

2. Choose initialization method

Select an initialization method for the values of your neural network parameters¹.

☐ Zero ☐ Too small ☒ Appropriate ☐ Too large

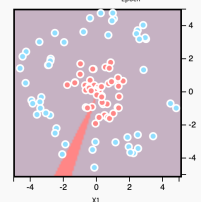
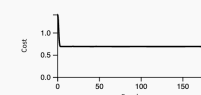


Select whether to visualize the weights or gradients of the network above.

☐ Weight ☒ Gradient

3. Train the network.

Observe the cost function and the decision boundary.



Initialization Goals

- The general goal of initialization techniques is to initialize the weights such that the variance of the activations are the same across every layer. This constant variance helps prevent the gradient from exploding or vanishing.

To help derive our initialization values, we will make the following **simplifying assumptions**:

- Weights and inputs are centered at zero
- Weights and inputs are independent and identically distributed
- Biases are initialized as zeros

Xavier + He Initialization

● Xavier Initialization

- Designed for the tanh activation function
- Utilizes assumption that tanh is approx. linear for small inputs
- $W_{i,j}^{[l]} = \mathcal{N}\left(0, \frac{1}{n^{[l-1]}}\right)$

● He Initialization

- Designed for the ReLU activation function
- Also known as Kaiming Initialization
- Used by default by PyTorch for Linear layer

$$\begin{aligned} \text{Var}(a_i^{[\ell-1]}) &= \text{Var}(a_i^{[\ell]}) \\ &= \text{Var}(z_i^{[\ell]}) \quad \longleftarrow \text{linearity of tanh around zero } \tanh(z) \approx z \\ &= \text{Var}\left(\sum_{j=1}^{n^{[\ell-1]}} w_{ij}^{[\ell]} a_j^{[\ell-1]}\right) \\ &= \sum_{j=1}^{n^{[\ell-1]}} \text{Var}(w_{ij}^{[\ell]} a_j^{[\ell-1]}) \quad \longleftarrow \text{variance of independent sum } \text{Var}(X+Y) = \text{Var}(X) + \text{Var}(Y) \\ &= \sum_{j=1}^{n^{[\ell-1]}} E[w_{ij}^{[\ell]}]^2 \text{Var}(a_j^{[\ell-1]}) + E[a_j^{[\ell-1]}]^2 \text{Var}(w_{ij}^{[\ell]}) + \text{Var}(w_{ij}^{[\ell]}) \text{Var}(a_j^{[\ell-1]}) \quad \longleftarrow \text{variance of independent product } \text{Var}(XY) = E[X]^2 \text{Var}(Y) + E[Y]^2 \text{Var}(X) + \text{Var}(X) \text{Var}(Y) \\ &= n^{[\ell-1]} \text{Var}(w_{ij}^{[\ell]}) \text{Var}(a_j^{[\ell-1]}) \implies \text{Var}(W) = \frac{1}{n^{[\ell-1]}} \end{aligned}$$

Overfitting & Underfitting

Overfitting:

- High Variance
- Train performance significantly better than test performance
- Validation loss increasing after a certain point

Underfitting:

- High Bias
- Train and test loss nearly identical
- High loss, no signs of decreasing

Combating Overfitting & Underfitting

Overfitting:

- Increase amount of data
- Utilize data augmentation methods
- Decrease model size/complexity
- Implement early stopping
- Utilize regularization techniques

Underfitting:

- Increase model size and complexity
- Train for a longer period of time
- Reduce regularization techniques

L1 and L2 Regularization

Let's consider some cost function $J(w_1, \dots, w_l)$, a function of weight matrices $w_1, \dots, w_l$. Let's define the following two regularized cost functions:

$$J_{L_1}(w_1, \dots, w_l) = J(w_1, \dots, w_l) + \lambda \sum_{i=1}^l |w_i|$$
$$J_{L_2}(w_1, \dots, w_l) = J(w_1, \dots, w_l) + \lambda \sum_{i=1}^l ||w_i||^2$$

Let's derive the gradient updates for these cost functions.

The update for w_i when using J_{L_1} is:

$$w_i^{k+1} = w_i^k - \underbrace{\alpha \lambda \text{sign}(w_i)}_{L_1 \text{ penalty}} - \alpha \frac{\partial J}{\partial w_i}$$

The update for w_i when using J_{L_2} is:

$$w_i^{k+1} = w_i^k - \underbrace{2\alpha \lambda w_i}_{L_2 \text{ penalty}} - \alpha \frac{\partial J}{\partial w_i}$$

Define a weight matrix $\mathbf{w} \in \mathbb{R}^{m \times n}$

Denote the element in row i and column j as $\mathbf{w}_{i,j}$

$$|\mathbf{w}| = \sum_{i=1}^m \sum_{j=1}^n |\mathbf{w}_{i,j}|$$
$$||\mathbf{w}|| = \sqrt{\sum_{i=1}^m \sum_{j=1}^n \mathbf{w}_{i,j}^2}$$
$$||\mathbf{w}||^2 = \sum_{i=1}^m \sum_{j=1}^n \mathbf{w}_{i,j}^2$$

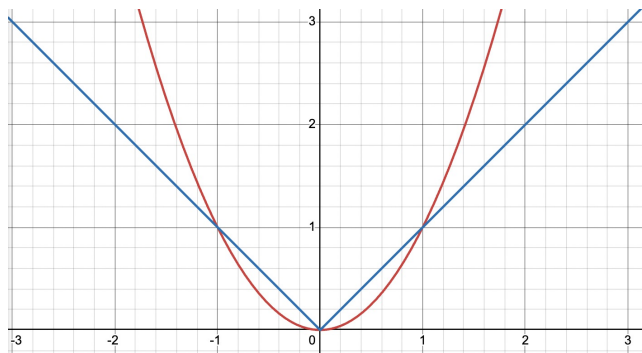
$$f(x) = |x| \Rightarrow \frac{\partial f}{\partial x} = ?$$

$$f(x) = \begin{cases} x & \text{if } x \geq 0 \\ -x & \text{otherwise} \end{cases}$$

$$\frac{\partial f}{\partial x} = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{otherwise} \end{cases} = \text{sign}(x)$$

$$f(x) = x^2 \Rightarrow \frac{\partial f}{\partial x} = 2x$$

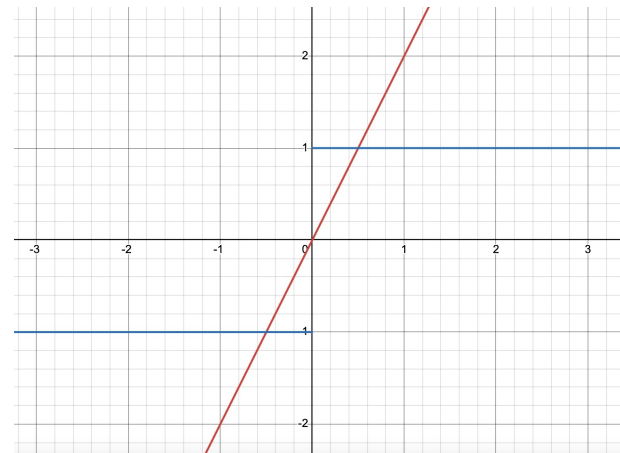
L1 and L2 Regularization Intuition



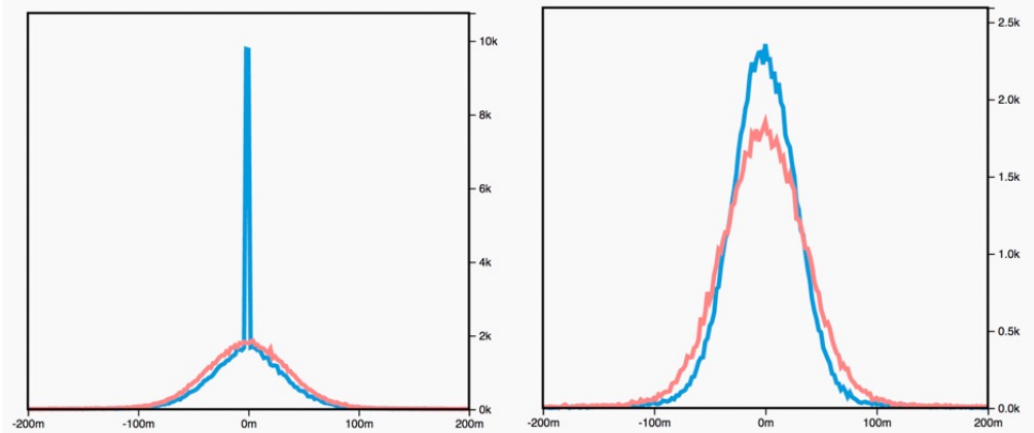
L2 Penalty (Red) vs L1 Penalty (Blue)

L1 Regularization generally pushes many values to 0 (sparser weights)

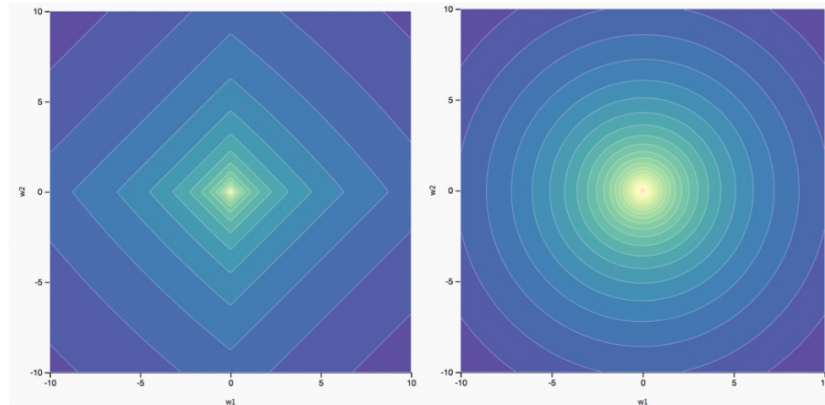
L2 Regularization generally more heavily impacts larger values, and doesn't force many values to 0



L2 Grad. (Red) vs L1 Grad. (Blue)



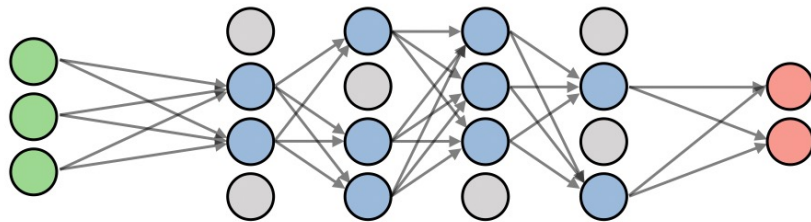
Histogram of weight values for unregularized (red), L1 regularized (blue, left), L2 regularized (blue, right)



Heatmap of L1 penalty (left) and L2 penalty (right) in a 2-parameter setting.

Dropout

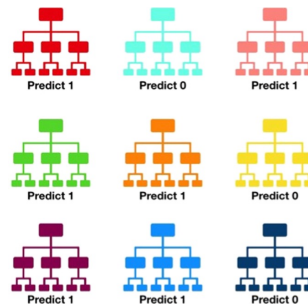
- With some specified p , drop any given neuron of the neural network with probability p during a training iteration. Do this for each neuron independently.
- Only applies during training. During testing, all neurons are turned on and dropout is disabled.
- Helps prevent overfitting by making the model more generalizable. Doesn't improve model's capacity, rather focuses on generalizability of the model.



Intuitive Explanation:

We are training a forest of models to make our evaluation.

During testing, we bring all these uniquely trained models together and make an aggregate evaluation.



Avoids single neurons becoming overly important and helps equalize capacity/power of each neuron